

Guide de déploiement Athenis

De zéro à la production — pour débutant total

Version 2026

Plateforme financière, comptable et ESG

SYSCOHADA / PCG France

Document confidentiel

Guide complet de déploiement Athenis

De zéro à la production — pour débutant total

Auteur : Équipe Athenis

Cible : utilisateur sans connaissance technique préalable

Durée totale estimée : **3 à 5 heures**, étalées sur 24 h (le DNS prend du temps à se propager).

Sommaire

1. Vue d'ensemble : ce que tu vas faire
 2. Phase 0 : préparation de ton poste local
 3. Phase 1 : acheter un nom de domaine chez OVH
 4. Phase 2 : louer un serveur (VPS) chez OVH
 5. Phase 3 : première connexion au serveur (SSH)
 6. Phase 4 : sécuriser et préparer le serveur
 7. Phase 5 : pointer le domaine vers le serveur (DNS)
 8. Phase 6 : récupérer le code Athenis depuis GitHub
 9. Phase 7 : générer les secrets et configurer le `.env`
 10. Phase 8 : obtenir un certificat HTTPS (Let's Encrypt)
 11. Phase 9 : démarrer Athenis avec Docker Compose
 12. Phase 10 : appliquer les migrations + créer le premier admin
 13. Phase 11 : configurer l'envoi d'emails (Brevo)
 14. Phase 12 : tester la solution
 15. Phase 13 : sauvegardes automatiques de la base
 16. Phase 14 : surveillance (UptimeRobot, Sentry)
 17. Phase 15 : mettre à jour Athenis quand une nouvelle version sort
 18. Phase 16 : distribuer Athenis en application — PWA, Desktop, Mobile
 19. Phase 17 : dépannage — problèmes fréquents
 20. Annexes
 - A. Glossaire des termes techniques
 - B. Checklist finale avant mise en production
 - C. Commandes Linux essentielles
 - D. Coûts mensuels récapitulatifs
 - E. Tableau comparatif Web / Desktop / Mobile
-

1. Vue d'ensemble {#1-vue-densemble}

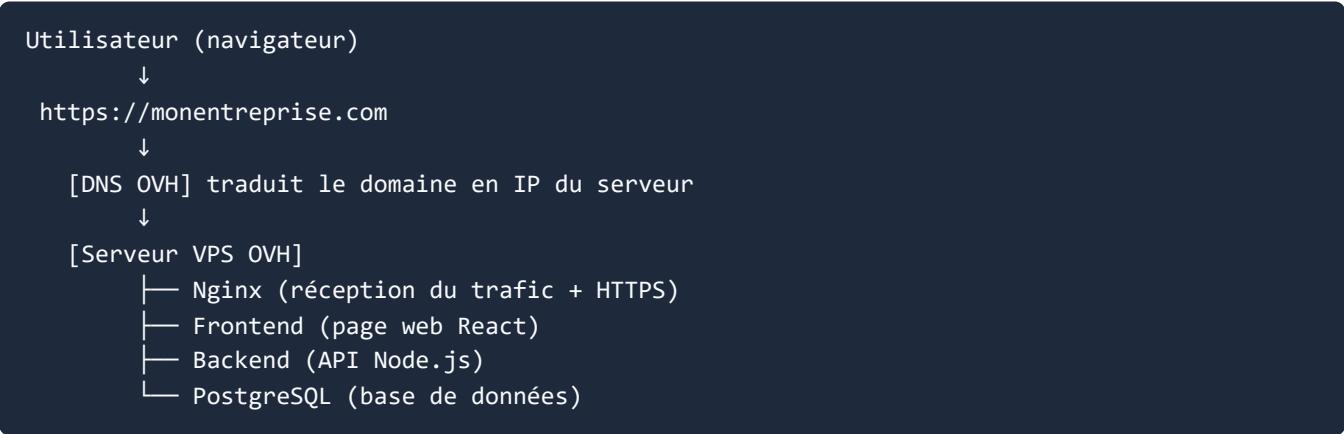
Athenis est une application web (consultable dans n'importe quel navigateur) composée de **trois éléments** qui doivent fonctionner ensemble :

Élément	Rôle	Technologie
Frontend	L'interface visuelle que voient tes utilisateurs	React + Vite (HTML/CSS/JS)
Backend	Le cerveau qui calcule, sauvegarde et applique les règles	Node.js + Express
Base de données	Le stockage permanent des factures, paies, comptes...	PostgreSQL

Pour rendre Athenis accessible sur Internet, tu vas :

1. **Acheter un nom de domaine** (par exemple `monentreprise.com`) — c'est l'adresse que tes utilisateurs taperont.
2. **Louer un serveur (VPS)** — un ordinateur en location dans un datacenter, allumé 24h/24.
3. **Pointer le domaine vers le serveur** — pour qu'en tapant `monentreprise.com`, on arrive sur ton serveur.
4. **Installer Athenis sur le serveur** — via Docker, qui fait tourner les trois éléments automatiquement.
5. **Activer HTTPS** (le cadenas vert) — gratuit grâce à Let's Encrypt.

Schéma simplifié



Coûts mensuels estimés

Service	Coût	Périodicité
Domaine <code>.com</code> ou <code>.fr</code>	7 à 10 €	par an
VPS OVH Starter (2 GB RAM, 40 GB SSD)	6 €	par mois
Email SMTP Brevo (300 envois/jour)	0 €	gratuit
Surveillance UptimeRobot	0 €	gratuit
Sauvegards (incluses dans le VPS)	0 €	gratuit
TOTAL	~6 €/mois + 8 €/an	

2. Phase 0 : préparation de ton poste {#2-phase-0--preparation-de-ton-poste}

Avant de commencer, installe ces outils **sur ton ordinateur** (Windows, Mac ou Linux). Ils servent à communiquer avec le serveur distant.

2.1 — Sur Windows

Installe Windows Terminal

1. Ouvre le **Microsoft Store**. 2. Cherche "**Windows Terminal**". 3. Clique **Installer** (gratuit, officiel Microsoft).

Installe Git

1. Va sur <https://git-scm.com/download/win> 2. Télécharge l'installateur 64-bit. 3. Exécute le `.exe` et clique **Suivant** partout (les options par défaut sont bonnes). 4. À la fin, ouvre **Windows Terminal** et tape :

```
git --version
```

Tu dois voir un numéro de version (ex. `git version 2.45.0`).

Installe un éditeur de texte : Notepad++

1. Va sur <https://notepad-plus-plus.org/downloads/> 2. Télécharge la version 64-bit. 3. Installe-la.

Notepad++ permet d'éditer les fichiers de configuration sans erreur d'encodage (contrairement au Bloc-notes Windows).

2.2 — Sur macOS

Ouvre Terminal

- Ouvre **Spotlight** (⌘ + Espace), tape "Terminal", appuie sur Entrée.

Installe Homebrew (gestionnaire de logiciels)

Dans le Terminal :

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Saisis ton mot de passe Mac quand demandé. Patiente ~5 minutes.

Installe Git et un éditeur

```
brew install git
```

Pour l'éditeur, **TextEdit** suffit (déjà installé), mais **VS Code** est mieux : télécharge sur <https://code.visualstudio.com/>.

2.3 — Sur Linux (Ubuntu/Debian)

Dans un terminal :

```
sudo apt update  
sudo apt install -y git curl nano
```

3. Phase 1 : acheter un nom de domaine chez OVH {#3-phase-1--acheter-un-nom-de-domaine}

Durée : 15 minutes

Le nom de domaine est l'adresse de ton site (ex. `athenis-comptabilite.com`). C'est lui que tes utilisateurs tapent dans le navigateur.

3.1 — Créer un compte OVH

1. Va sur <https://www.ovh.com/manager/> et clique **Créer un compte**.
2. Remplis :
 - Type de compte : **Professionnel** (recommandé pour avoir une facture)
 - Email valide (sera ton identifiant)
 - Mot de passe fort
 - Adresse postale réelle (vérifications anti-fraude)
3. Valide ton email en cliquant sur le lien reçu.

3.2 — Acheter le domaine

1. Connecte-toi sur <https://www.ovh.com/fr/domaines/>
2. Dans la barre de recherche, tape le nom souhaité, par exemple :
 - `monentreprise.com` (international, ~9 €/an)
 - `monentreprise.fr` (France, ~7 €/an)
 - `monentreprise.cm` (Cameroun, ~120 €/an, vérification d'identité)
3. Clique **Vérifier la disponibilité**.
4. Si vert : clique **Ajouter au panier**. Sinon, essaie une variation.
5. À l'étape suivante, **DÉCOCHE TOUT CE QUI EST OPTIONNEL** (DNS Anycast, MX Plan, etc.). Tu n'en as pas besoin pour démarrer.
6. Valide les conditions et paie par carte bancaire.

3.3 — Attendre l'activation

Le domaine est généralement actif sous **5 à 30 minutes**. Tu reçois un email "Votre domaine est actif".

À noter : Garde précieusement ton **identifiant client OVH** (ex. **ab123456-ovh**) et ton mot de passe. Tu en auras besoin pour gérer le DNS plus tard.

4. Phase 2 : louer un VPS chez OVH {#4-phase-2--louer-un-vps-ovh}

Durée : 20 minutes + 10 à 30 min d'installation automatique

Un VPS (Virtual Private Server) est un ordinateur Linux en location dans un datacenter OVH, accessible en permanence depuis Internet.

4.1 — Choisir l'offre

1. Connecte-toi sur <https://www.ovh.com/manager/>
2. Va dans **Bare Metal Cloud** → **VPS** ou directement <https://www.ovhcloud.com/fr/vps/>
3. Choisis **VPS Value** ou **VPS Comfort** :

Offre	RAM	CPU	Stockage	Prix HT	Adapté pour
Starter	2 GB	1 vCore	40 GB SSD	3,99 €/mois	1-5 utilisateurs (test)
Value	4 GB	2 vCores	80 GB SSD	5,99 €/mois	10-30 utilisateurs (recommandé)
Essential	8 GB	4 vCores	160 GB SSD	9,99 €/mois	30-100 utilisateurs

Recommandation : commence avec VPS Value (5,99 €/mois). Tu peux upgrader plus tard sans perdre les données.

4.2 — Configurer le VPS

1. Clique **Commander** sur l'offre Value.
2. Engagement : choisis **1 mois** (renouvelable, plus de flexibilité).
3. Datacenter : **Gravelines (France)** ou **Strasbourg** (proche de tes utilisateurs).
4. **Système d'exploitation** : sélectionne **Ubuntu 24.04 LTS** (le plus stable, supporté jusqu'en 2029).
5. **Sauvegarde automatique** : coche **"Backup auto - 2 GB"** (~2 €/mois supplémentaires — fortement recommandé).
6. **Options de sécurité** : décoche toutes les options payantes (anti-DDoS de base inclus gratuit).

4.3 — Finaliser la commande

1. Vérifie le résumé.
2. Paie par carte bancaire.
3. **Tu reçois un email avec :**

- **L'adresse IP de ton VPS** (4 nombres séparés par des points, ex. `51.91.234.123`)
- **L'utilisateur** (souvent `ubuntu` ou `root`)
- **Un lien pour définir le mot de passe initial**

PRENDS NOTE de l'IP et de l'utilisateur, tu en auras besoin dans 5 minutes.

4.4 — Définir le mot de passe SSH

1. Clique sur le lien reçu par email.
2. Choisis un **mot de passe très fort** (16 caractères, mélange majuscules/minuscules/chiffres/symboles). Note-le dans un gestionnaire de mots de passe (Bitwarden, KeePass, 1Password).
3. Confirme.

4.5 — Attendre la fin de l'installation

L'installation d'Ubuntu prend **10 à 30 minutes**. Tu reçois un nouvel email "**Votre VPS est prêt**".

5. Phase 3 : première connexion au serveur (SSH) {#5-phase-3--connexion-ssh}

Durée : 10 minutes

SSH est le protocole qui permet de contrôler le serveur à distance via un terminal.

5.1 — Connexion depuis Windows

1. Ouvre **Windows Terminal**.
2. Tape (remplace `51.91.234.123` par TON IP) :

```
ssh ubuntu@51.91.234.123
```

3. Le système te demande :

```
The authenticity of host '51.91.234.123' can't be established.  
Are you sure you want to continue connecting (yes/no)?
```

Tape `yes` et appuie sur Entrée.

4. Saisis le mot de passe défini à l'étape 4.4 (rien ne s'affiche pendant la saisie — c'est normal).
5. Tu dois voir un prompt comme :

```
ubuntu@vps-12345:~$
```

Bravo, tu es connecté à ton serveur !

5.2 — Connexion depuis macOS / Linux

Identique à Windows, mais dans **Terminal** (macOS) ou n'importe quel terminal Linux :

```
ssh ubuntu@51.91.234.123
```

5.3 — Si la connexion échoue

Erreur	Solution
<code>Permission denied</code>	Mauvais mot de passe ou mauvais utilisateur. Vérifie l'email d'OVH.
<code>Connection refused</code>	Le VPS n'est peut-être pas encore prêt. Attends 10 minutes et réessaie.
<code>Host key verification failed</code>	Tape <code>ssh-keygen -R 51.91.234.123</code> puis réessaie.

6. Phase 4 : sécuriser et préparer le serveur

{#6-phase-4--securiser-le-serveur}

Durée : 30 minutes

Tu vas effectuer plusieurs étapes pour rendre le serveur sécurisé et prêt à accueillir Athenis.

⚠ *Toutes les commandes ci-dessous se tapent SUR LE SERVEUR (via SSH), pas sur ton poste local.*

6.1 — Mettre à jour le système

```
sudo apt update && sudo apt upgrade -y
```

Si le système te demande de redémarrer des services : tape **1** puis Entrée (option par défaut).

6.2 — Créer un utilisateur dédié (sans droits root permanents)

Pour des raisons de sécurité, on évite de tout faire en root.

```
sudo adduser athenis
```

Réponds aux questions :

- Mot de passe : choisis un mot de passe fort (différent du mot de passe SSH).
- Full Name, Room Number, etc. : appuie sur Entrée (pas obligatoire).
- Is the information correct? : tape **Y** et Entrée.

Donne les droits sudo (administrateur) à cet utilisateur :

```
sudo usermod -aG sudo athenis
```

6.3 — Installer Docker (le plus important)

Docker fait tourner Athenis dans des "conteneurs" isolés. Une seule commande l'installe :

```
curl -fsSL https://get.docker.com | sudo sh
```

Patiente 1-2 minutes. À la fin, ajoute ton utilisateur au groupe docker :

```
sudo usermod -aG docker athenis
sudo usermod -aG docker ubuntu
```

6.4 — Installer les outils supplémentaires

```
sudo apt install -y git curl nano htop ufw fail2ban
```

Outil	Rôle
git	Récupérer le code d'Athenis depuis GitHub
nano	Éditeur de texte simple en ligne de commande
htop	Surveiller les ressources du serveur
ufw	Pare-feu simple
fail2ban	Bloque automatiquement les tentatives de connexion abusives

6.5 — Configurer le pare-feu (ufw)

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp    # HTTPS
sudo ufw enable
```

Tape **y** pour confirmer. Vérifie :

```
sudo ufw status
```

Tu dois voir :

```
Status: active
22/tcp      ALLOW      Anywhere
80/tcp      ALLOW      Anywhere
443/tcp     ALLOW      Anywhere
```

6.6 — Configurer fail2ban (protection contre les tentatives de force brute SSH)

```
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
```

6.7 — Tester Docker

```
docker run hello-world
```

Tu dois voir un message "**Hello from Docker!**" — c'est bon.

6.8 — Se reconnecter en tant qu'utilisateur `athenis`

Quitte la session actuelle :

```
exit
```

Reconnecte-toi en tant qu' `athenis` :

```
ssh athenis@51.91.234.123
```

À partir de maintenant, **toutes les commandes se font sous l'utilisateur `athenis`** .

7. Phase 5 : pointer le domaine vers le serveur (DNS) {#7-phase-5--pointer-le-dns}

Durée : 10 minutes + délai de propagation 1 à 24 heures

7.1 — Accéder à la zone DNS OVH

1. Connecte-toi sur <https://www.ovh.com/manager/>
2. **Web Cloud** → **Domaines** → **ton-domaine.com** → **Zone DNS**

7.2 — Créer les enregistrements A

Tu vas dire à Internet : "le domaine `ton-domaine.com` est servi par le serveur d'IP `51.91.234.123`".

1. Clique **Ajouter une entrée**.
2. Sélectionne **type A**.
3. Remplis :
 - **Sous-domaine** : laisse vide (= domaine racine)
 - **TTL** : par défaut (3600)
 - **Cible** : `51.91.234.123` (TON IP)
4. Valide.

Recommence pour le sous-domaine `www` :

1. **Ajouter une entrée** → **A**.
2. **Sous-domaine** : `www`.
3. **Cible** : `51.91.234.123`.
4. Valide.

7.3 — Récapitulatif des entrées DNS

Tu dois avoir au minimum :

A	@	51.91.234.123	(ton-domaine.com)
A	www	51.91.234.123	(www.ton-domaine.com)

7.4 — Vérifier la propagation

La propagation DNS prend **1 à 24 heures** (généralement < 1 heure). Vérifie avec :


```
nslookup ton-domaine.com
```

Si la réponse contient ton IP de serveur, c'est bon. Sinon, patiente.

Tu peux aussi vérifier sur <https://dnschecker.org/> — entre ton domaine et regarde si tous les drapeaux mondiaux sont verts.

8. Phase 6 : récupérer le code Athenis depuis GitHub {#8-phase-6--cloner-athenis}

Durée : 5 minutes

Sur le serveur (connecté en SSH en tant qu' `athenis`) :

8.1 — Cloner le dépôt

```
cd /home/athenis
git clone https://github.com/ulrichmouafo/athenis.git
cd athenis
```

***Remarque** : si le dépôt est **privé**, tu devras configurer un token GitHub. Voir Annexe D.*

8.2 — Vérifier la structure

```
ls -la
```

Tu dois voir des dossiers `apps/` , `infra/` , `package.json` , `DEPLOY.md` , etc.

9. Phase 7 : générer les secrets et configurer le `.env` `{#9-phase-7--configurer-env}`

Durée : 15 minutes

Le fichier `.env` contient toutes les **clés secrètes** qu'Athenis utilise pour chiffrer les données et sécuriser les connexions. Il ne doit jamais être partagé ou commité sur Git.

9.1 — Copier le modèle

```
cp .env.example .env
```

9.2 — Générer des secrets aléatoires forts

Sur le serveur, exécute :

```
echo "JWT_SECRET=$(openssl rand -base64 48)"
echo "JWT_REFRESH_SECRET=$(openssl rand -base64 48)"
echo "ENCRYPTION_KEY=$(openssl rand -base64 32 | head -c 32)"
echo "POSTGRES_PASSWORD=$(openssl rand -base64 24)"
```

Copie les 4 valeurs générées dans un fichier texte temporaire sur ton poste local (pas dans Athenis).

9.3 — Éditer le `.env`

```
nano .env
```

Modifie les lignes suivantes en collant tes valeurs générées :

```
# Database – utilise le mot de passe fort généré ci-dessus
DATABASE_URL=postgresql://athenis:LE_MOT_DE_PASSE_GENERE@db:5432/athenis_db
POSTGRES_PASSWORD=LE_MOT_DE_PASSE_GENERE
```

JWT – secrets aléatoires 48+ caractères

```
JWT_SECRET=LA_VALEUR_GENEREE_1
JWT_REFRESH_SECRET=LA_VALEUR_GENEREE_2
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
```

Chiffrement – EXACTEMENT 32 caractères

ENCRYPTION_KEY=LA_VALEUR_GENEREE_3

Application

NODE_ENV=production

PORT=3001

FRONTEND_URL=https://ton-domaine.com # ← remplace par ton vrai domaine

Rate limiting

```
RATE_LIMIT_WINDOW_MS=900000  
RATE_LIMIT_MAX=100
```

Pour SMTP, Sentry, PostHog : laisse vides pour le moment, on configurera dans les phases suivantes.

Sauvegarde : **Ctrl+O** puis Entrée, puis **Ctrl+X** .

9.4 — Sécuriser le `.env`

```
chmod 600 .env
```

Seul ton utilisateur peut maintenant lire ce fichier.

10. Phase 8 : obtenir un certificat HTTPS (Let's Encrypt) {#10-phase-8--ssl-letsencrypt}

Durée : 10 minutes

HTTPS (le cadenas vert) est **obligatoire** pour Athenis. C'est gratuit grâce à Let's Encrypt.

10.1 — Vérifier que le DNS pointe bien

Sur ton serveur :

```
ping -c 3 ton-domaine.com
```

Tu dois voir l'IP de ton serveur. **Si ce n'est pas le cas, attends la propagation DNS avant de continuer.**

10.2 — Installer Certbot via Docker

```
mkdir -p infra/nginx/certs infra/nginx/certbot-www
```

10.3 — Première obtention du certificat

Arrête tout service éventuel sur les ports 80/443 :

```
docker compose -f infra/docker-compose.yml down 2>/dev/null
```

Génère le certificat (remplace `ton-domaine.com` par TON domaine et `toi@email.com` par ton email) :


```
docker run --rm -it \
-v "$(pwd)/infra/nginx/certs:/etc/letsencrypt" \
-v "$(pwd)/infra/nginx/certbot-www:/var/www/certbot" \
-p 80:80 \
certbot/certbot certonly --standalone \
--preferred-challenges http \
-d ton-domaine.com \
-d www.ton-domaine.com \
--email toi@email.com \
--agree-tos --no-eff-email
```

Patiente. À la fin tu dois voir :

```
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/ton-domaine.com/fullchain.pem
```

10.4 — Vérifier les certificats

```
sudo ls infra/nginx/certs/live/ton-domaine.com/
```

Tu dois voir : `cert.pem` , `chain.pem` , `fullchain.pem` , `privkey.pem` .

10.5 — Configurer Nginx pour utiliser ces certificats

Édite le fichier de configuration Nginx fourni :

```
nano infra/nginx/nginx.conf
```

Cherche les occurrences de `votre-domaine.com` ou `example.com` et remplace par `ton-domaine.com` .
Sauvegarde.

11. Phase 9 : démarrer Athenis avec Docker Compose {#11-phase-9--demarrer-docker-compose}

Durée : 15 à 30 minutes (premier build long)

11.1 — Construire les images Docker

```
docker compose -f infra/docker-compose.yml build
```

Cette étape **dure 10 à 20 minutes** (téléchargement de Node, PostgreSQL, construction du frontend). C'est normal.

11.2 — Démarrer les services

```
docker compose -f infra/docker-compose.yml up -d
```

L'option `-d` veut dire "en arrière-plan" (le terminal reste utilisable).

11.3 — Vérifier le démarrage

```
docker compose -f infra/docker-compose.yml ps
```

Tu dois voir 4 services :

NAME	STATUS
athenis-db	Up (healthy)
athenis-backend	Up
athenis-frontend	Up
athenis-nginx	Up

11.4 — Consulter les logs en cas de problème

```
docker compose -f infra/docker-compose.yml logs -f
```

Ctrl+C pour sortir.

Pour un service spécifique :

```
docker compose -f infra/docker-compose.yml logs backend  
docker compose -f infra/docker-compose.yml logs db
```

12. Phase 10 : appliquer les migrations + créer le premier admin {#12-phase-10--migrations-admin}

Durée : 10 minutes

12.1 — Appliquer les migrations Prisma

Les migrations créent les tables PostgreSQL :

```
docker compose -f infra/docker-compose.yml exec backend npx prisma migrate deploy
```

Tu dois voir :

```
All migrations have been successfully applied.
```

12.2 — Vérifier que les tables sont créées

```
docker compose -f infra/docker-compose.yml exec db psql -U athenis -d athenis_db -c "\dt"
```

Liste des tables visibles (users, companies, invoices, employees...).

12.3 — Créer le premier compte super-admin

```

docker compose -f infra/docker-compose.yml exec backend node -e "
const { PrismaClient } = require('@prisma/client')
const bcrypt = require('bcryptjs')
const p = new PrismaClient()
async function main() {
  const password = 'CHANGE_MOI_MAINTENANT!2026'
  const hash = await bcrypt.hash(password, 12)
  const user = await p.user.create({
    data: {
      email: 'admin@ton-domaine.com',
      passwordHash: hash,
      firstName: 'Admin',
      lastName: 'Principal',
      accountType: 'COMPANY',
      role: 'ADMIN',
      platformRole: 'SUPER_ADMIN',
      isActive: true,
      atheisNumber: 'ATH-E-00001',
    },
  })
  console.log('Admin créé :', user.email)
  console.log('Mot de passe :', password)
  console.log('CHANGE LE MOT DE PASSE A LA PREMIERE CONNEXION !')
  await p.$disconnect()
}
main().catch(console.error)
"

```

Note bien le mot de passe affiché et change-le immédiatement après la première connexion.

13. Phase 11 : configurer l'envoi d'emails (Brevo) {#13-phase-11--smtp-brevo}

Durée : 20 minutes

Athenis envoie des emails (bulletins de paie, factures, notifications). On utilise **Brevo** (ex-Sendinblue) — 300 emails/jour gratuits.

13.1 — Créer un compte Brevo

1. Va sur <https://www.brevo.com/fr/>
2. Clique "**Commencer gratuitement**".
3. Email + mot de passe + numéro de téléphone (vérification SMS).
4. Choisis le **plan Gratuit** (300 emails/jour).

13.2 — Vérifier ton domaine d'envoi

Brevo demande de prouver que tu es propriétaire du domaine.

1. Dans Brevo → **Senders & IP** → **Domains**.
2. Clique "**Add a domain**" → tape `ton-domaine.com`.
3. Brevo te donne **3 enregistrements DNS** (TXT, DKIM, SPF) à ajouter.
4. Va dans OVH **Domaines** → `ton-domaine.com` → **Zone DNS**.
5. **Ajoute les 3 enregistrements** un par un (type **TXT** ou **CNAME** selon Brevo).
6. Retourne sur Brevo et clique "**Verify**". Patiente 10-30 minutes pour la propagation.

13.3 — Générer une clé SMTP

1. Brevo → **SMTP & API** → **SMTP**.
2. Clique "**Generate a new SMTP key**".
3. Donne un nom (ex. `Athenis Production`).
4. **Copie immédiatement la clé** (elle ne sera plus affichée).

Tu obtiens :

- **SMTP server** : `smtp-relay.brevo.com`
- **Port** : `587`

- **Login** : ton email Brevo
- **SMTP key** : (la clé copiée)

13.4 — Renseigner dans `.env`

Sur le serveur :

```
cd ~/athenis  
nano .env
```

Mets à jour :

```
SMTP_HOST=smtp-relay.brevo.com  
SMTP_PORT=587  
SMTP_SECURE=false  
SMTP_USER=ton-email@brevo-compte.com  
SMTP_PASS=la_cle_smtp_copiee  
SMTP_FROM=Athenis <noreply@ton-domaine.com>
```

Sauvegarde (`Ctrl+O` , Entrée, `Ctrl+X`).

13.5 — Redémarrer le backend pour prendre en compte

```
docker compose -f infra/docker-compose.yml restart backend
```

14. Phase 12 : tester la solution {#14-phase-12--tester}

Durée : 10 minutes

14.1 — Tester le frontend

Ouvre ton navigateur et va sur <https://ton-domaine.com>.

Tu dois voir la page de connexion d'Athenis avec le cadenas vert dans la barre d'adresse.

14.2 — Se connecter

- Email : `admin@ton-domaine.com`
- Mot de passe : celui affiché à la phase 10.3

14.3 — Vérifications

Vérification	Comment
Le cadenas est vert	Clic sur le cadenas → certificat valide Let's Encrypt
Le tableau de bord se charge	Les KPIs apparaissent (CA, marge, etc.)
Les modules sont accessibles	Clic Gestion, Comptabilité, RH...
Les emails marchent	Settings → Mon profil → "Tester l'envoi d'email"

14.4 — Si une erreur apparaît

Va voir les logs :

```
docker compose -f infra/docker-compose.yml logs -f backend
```

Voir aussi la section **18. Phase 16 : dépannage**.

15. Phase 13 : sauvegardes automatiques de la base {#15-phase-13--sauvegardes}

Durée : 15 minutes

OVH inclut une sauvegarde quotidienne de tout le VPS (~2 €/mois supplémentaires). En plus, on configure une sauvegarde **de la base de données** spécifiquement.

15.1 — Créer un dossier de sauvegardes

```
mkdir -p /home/athenis/backups
```

15.2 — Créer le script de backup

```
nano /home/athenis/backup-db.sh
```

Colle ce contenu :

```
#!/bin/bash
```

Sauvegarde quotidienne de PostgreSQL

```
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR=/home/athenis/backups
KEEP_DAYS=30
```

Dump

```
docker compose -f /home/athenis/athenis/infra/docker-compose.yml exec -T db \  
pg_dump -U athenis athenis_db | gzip > "${BACKUP_DIR}/athenis_${TIMESTAMP}.sql.gz"
```

Supprime les sauvegardes de plus de 30

```
find "${BACKUP_DIR}" -name 'athenis_*.sql.gz' -mtime +${KEEP_DAYS} -delete  
echo "[$(date)] Backup réussi : ${BACKUP_DIR}/athenis_${TIMESTAMP}.sql.gz"
```

Sauvegarde et rends exécutable :

```
chmod +x /home/athenis/backup-db.sh
```

15.3 — Tester le script

```
/home/athenis/backup-db.sh  
ls -lh /home/athenis/backups/
```

Tu dois voir un fichier `athenis_AAAAMMJJ_HHMMSS.sql.gz` .

15.4 — Planifier l'exécution quotidienne (cron)

```
crontab -e
```

Choisis `1` (nano) si demandé. Ajoute en bas :

```
0 3 * /home/athenis/backup-db.sh >> /home/athenis/backups/backup.log 2>&1
```

Cette ligne exécute le script **tous les jours à 3h du matin**.

Sauvegarde (`Ctrl+O` , Entrée, `Ctrl+X`).

15.5 — Restaurer un backup (si besoin un jour)

Pour restaurer (en cas de problème) :

```
gunzip -c /home/athenis/backups/athenis_AAAAMMJJ_HHMMSS.sql.gz | \  
docker compose -f /home/athenis/athenis/infra/docker-compose.yml exec -T db \  
psql -U athenis -d athenis_db
```

⚠ Ceci écrase la base actuelle !

16. Phase 14 : surveillance (UptimeRobot, Sentry) {#16-phase-14--monitoring}

Durée : 30 minutes

16.1 — UptimeRobot (alerte si le site tombe)

1. Va sur <https://uptimerobot.com/> → Inscription gratuite.
2. **Add New Monitor** → **HTTPS**.
3. **Friendly Name** : `Athenis`.
4. **URL** : `https://ton-domaine.com/api/health`.
5. **Monitoring Interval** : 5 minutes.
6. **Alert Contacts** : ajoute ton email + numéro de téléphone (SMS gratuit).
7. **Create Monitor**.

Si Athenis tombe, tu reçois un email/SMS dans les 5 minutes.

16.2 — Sentry (collecte des erreurs JavaScript / Node)

1. Va sur <https://sentry.io/> → Sign Up (gratuit).
2. **Create Project** → **Node.js** → nom `athenis-backend`.
3. Copie le **DSN** affiché (ex. `https://abc@xxx.ingest.sentry.io/123456`).
4. Idem pour un projet **React** : nom `athenis-frontend`, DSN noté.

Sur le serveur, édite `.env` :

```
nano /home/athenis/athenis/.env
```

Ajoute :

```
SENTRY_DSN=https://abc@xxx.ingest.sentry.io/123456
VITE_SENTRY_DSN=https://def@xxx.ingest.sentry.io/789012
```

Redémarre :

```
cd ~/athenis
docker compose -f infra/docker-compose.yml restart
```

16.3 — PostHog (analytics — optionnel)

1. Va sur <https://eu.posthog.com/> → Sign Up.
2. **Create Project** → copie la **Project API Key**.
3. Ajoute dans `.env` :

```
VITE_POSTHOG_KEY=phc_xxx  
VITE_POSTHOG_HOST=https://eu.i.posthog.com
```

4. Redémarre.
-

17. Phase 15 : mettre à jour Athenis {#17-phase-15--maj}

Quand une nouvelle version d'Athenis sort sur GitHub :

```
cd /home/athenis/athenis
```

1. Récupère les changements

```
git pull origin main
```

2. Reconstruct les images

```
docker compose -f infra/docker-compose.yml build
```

3. Applique les migrations

```
docker compose -f infra/docker-compose.yml exec backend npx prisma migrate deploy
```


4. Redémarrage

```
docker compose -f infra/docker-compose.yml up -d
```

5. Vérifie les logs

```
docker compose -f infra/docker-compose.yml logs --tail=50
```

Toujours faire une sauvegarde manuelle AVANT :

```
/home/athenis/backup-db.sh
```

17.1 — Renouveler le certificat SSL (automatique tous les 3 mois)

Let's Encrypt expire après 90 jours. Pour le renouveler :

```
docker run --rm \
  -v "$(pwd)/infra/nginx/certs:/etc/letsencrypt" \
  certbot/certbot renew --quiet
docker compose -f infra/docker-compose.yml restart nginx
```

Automatise avec cron (mensuel) :

```
crontab -e
```

Ajoute :

```
0 4 1 cd /home/athenis/athenis && docker run --rm -v "$(pwd)/infra/nginx/certs:/etc/letsencry
```



18. Phase 16 : distribuer Athenis en application — PWA, Desktop, Mobile {#18-phase-16--distribution-apps}

Une fois ton serveur en ligne (phases 1 à 15), tu disposes d'**Athenis Web** consultable sur <https://ton-domaine.com>. Tu peux maintenant proposer aussi des **versions installables** sur les appareils de tes utilisateurs.

Quatre modes de distribution existent. Tu peux choisir un seul, plusieurs ou tous.

Mode	Plateforme	Statut	Effort
A. PWA (Web installable)	Tout navigateur + Android + iOS + Desktop	✅ Déjà actif	0
B. Desktop natif (Tauri)	Windows / macOS / Linux	⚙️ Configuré, à builder	2-4 h par OS
C. Mobile Android (Tauri Mobile)	Android 7+	⚙️ Configuré, à initialiser	4 h + signature
D. Mobile iOS (Tauri Mobile)	iPhone / iPad	⚙️ Configuré, Mac requis	8 h + 99 €/an Apple

18.A — Mode PWA (le plus simple, déjà actif) 🌐

C'est ce que tu as déjà déployé. N'importe quel utilisateur peut installer Athenis comme une appli en 2 clics depuis son navigateur.

Sur Android (Chrome, Edge, Brave)

1. L'utilisateur ouvre <https://ton-domaine.com> dans Chrome
2. Un bandeau « **Ajouter à l'écran d'accueil** » apparaît automatiquement
3. Clic → l'icône Athenis apparaît comme une vraie appli native
4. L'app s'ouvre en plein écran (sans barre de navigation), fonctionne hors-ligne en lecture

Sur iOS (Safari uniquement)

1. Ouvrir le site dans **Safari** (pas Chrome — iOS bloque les autres navigateurs)
2. Appuyer sur le bouton **Partager** (carré avec flèche vers le haut)
3. Faire défiler → « **Sur l'écran d'accueil** » → **Ajouter**
4. L'icône apparaît sur l'écran d'accueil

Sur Windows / macOS / Linux (Chrome / Edge)

1. Ouvrir le site dans Chrome ou Edge 2. **Icône « + » à droite de la barre d'adresse → Installer Athenis** 3. Raccourci créé sur le bureau et menu Démarrer

Avantages du mode PWA

- **✓ Aucune action de ta part** (déjà configuré via `manifest.json`)
- **✓ Mise à jour automatique** — tes utilisateurs ont toujours la dernière version
- **✓ Pas de magasin d'app** (pas de Google Play, pas d'App Store, pas de Microsoft Store)
- **✓ Aucun coût supplémentaire** (pas de compte développeur à 25 \$ ou 99 €/an)
- **✓ Fonctionne sur toutes les plateformes** avec un seul code

Limitations

- ⚠ Première ouverture nécessite Internet
- ⚠ Sur iOS, l'expérience est moins fluide (Safari limite certaines fonctions)
- ⚠ Pas dans le Play Store / App Store (les utilisateurs ne te trouvent pas par recherche)

Recommandation : commence avec ça pour 100 % de tes utilisateurs.

18.B — Application Desktop native (Windows / macOS / Linux)



Tauri est déjà configuré dans `apps/frontend/src-tauri/` . Tu obtiens un vrai **installateur** `.exe` , `.dmg` , ou `.AppImage` que tes utilisateurs téléchargent et installent.

Pré-requis communs

- **Node.js 18+** déjà installé sur ton poste (vérifie avec `node --version`)
- **Rust** : télécharge sur <https://rustup.rs/> → lance l'installateur, choisis **option 1**, patiente 10 minutes
- **Le code Athenis** sur ton poste (`git clone https://github.com/ulrichmouafo/athenis.git`)

Pré-requis Windows (uniquement pour build Windows)

Sur l'ordinateur où tu builds :

1. **Microsoft Edge WebView2 Runtime** : déjà installé sur Windows 10/11
2. **Microsoft Visual Studio Build Tools 2022** :
 - Télécharge sur <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
 - Pendant l'installation, coche « **Développement Desktop en C++** »
 - Patiente 15-30 min (~6 GB)

Build Windows (.exe + .msi)

Ouvre **PowerShell** ou **Windows Terminal** dans le dossier du projet :

```
cd C:\chemin\vers\athenis\apps\frontend
npm install
npm run tauri:build
```

Le build dure **10 à 20 minutes la première fois** (Rust compile tout). Les fois suivantes : 2-5 minutes.

Résultat dans `apps/frontend/src-tauri/target/release/bundle/` :

- `msi/Athenis_1.0.0_x64_en-US.msi` — installateur standard Windows
- `nsis/Athenis_1.0.0_x64-setup.exe` — installateur alternatif (plus rapide)

Pré-requis macOS (build .dmg)

Mac obligatoire (impossible depuis Windows). Sur le Mac :

```
# Installe Homebrew si absent
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Installe Xcode Command Line Tools

```
xcode-select --install
```

Installe Rust

```
brew install rustup
rustup-init
```

Puis :

```
cd ~/athenis/apps/frontend
npm install && npm run tauri:build
```

Résultat : `apps/frontend/src-tauri/target/release/bundle/dmg/Athenis_1.0.0_x64.dmg`

Pré-requis Linux (build .AppImage / .deb)

Sur Ubuntu 22.04+ :

```
sudo apt update
sudo apt install -y \
  libwebkit2gtk-4.1-dev \
  build-essential \
  curl wget file libxdo-dev \
  libssl-dev libayatana-appindicator3-dev librsvg2-dev

curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

Puis :

```
cd ~/athenis/apps/frontend
npm install && npm run tauri:build
```

Résultat :

- `appimage/athenis_1.0.0_amd64.AppImage` — universel Linux
- `deb/athenis_1.0.0_amd64.deb` — Ubuntu/Debian
- `rpm/athenis-1.0.0-1.x86_64.rpm` — Fedora/RHEL

Configurer l'URL du backend dans l'app

Par défaut, Tauri pointe sur `http://localhost:3001`. Pour la distribution, l'app doit savoir où trouver TON backend en production.

Édite `apps/frontend/src-tauri/tauri.conf.json` :

```
"security": {  
  "csp": "default-src 'self' tauri: https://tauri.localhost; connect-src 'self' ipc: http://ipc  
}
```

Remplace `http://localhost:3001` par `https://ton-domaine.com`.

Édite aussi `apps/frontend/src/lib/api.ts` (ou équivalent) pour pointer sur l'URL production :

```
const API_URL = import.meta.env.VITE_API_URL ?? 'https://ton-domaine.com/api'
```

Et `apps/frontend/.env.production` :

```
VITE_API_URL=https://ton-domaine.com/api
```

Re-builde après modification.

Distribution des installateurs

Méthode 1 — Hébergement direct

1. Mets les fichiers (`.msi` , `.dmg` , `.AppImage`) dans `apps/frontend/public/downloads/`
2. Re-déploie ton frontend
3. Ajoute des liens sur le site : [Télécharger pour Windows](#)

Méthode 2 — GitHub Releases (recommandée)

1. Va sur <https://github.com/ulrichmouafo/athenis/releases> → **Draft a new release**
2. Tag : `v1.0.0`
3. Upload les fichiers `.msi` , `.dmg` , `.AppImage`
4. Tes utilisateurs téléchargent depuis <https://github.com/ulrichmouafo/athenis/releases>

Méthode 3 — Signature numérique (pour éviter l'avertissement Windows SmartScreen)

- Acheter un certificat **EV Code Signing** (~250 €/an chez Sectigo, DigiCert, etc.)
- Signer le `.exe` avec `signtool.exe`
- Plus aucun avertissement à l'installation

Pour macOS : compte Apple Developer (99 €/an) + Apple Notarization automatique via Tauri.

Avantages Desktop natif

- ✓ Icône dans le menu Démarrer / Dock / Applications
- ✓ Fonctionne **complètement hors ligne** (toutes les pages déjà chargées)
- ✓ Notifications système natives
- ✓ Démarrage instantané (~50 Mo de RAM)
- ✓ Accès au système de fichiers (export Excel, scan facture, etc.)

Limitations

- ⚠ Mise à jour à la charge de l'utilisateur (ou intégrer Tauri Updater)
- ⚠ Un build différent pour Windows, Mac, Linux
- ⚠ Doit refaire un build à chaque nouvelle version Athenis

18.C — Application Android (Tauri Mobile)

Tauri 2 supporte Android nativement depuis fin 2024.

Pré-requis (sur ton PC Windows ou Mac)

1. Java JDK 17 :

- Windows : `winget install Microsoft.OpenJDK.17`
- macOS : `brew install openjdk@17`

2. **Android Studio** : <https://developer.android.com/studio> (~5 GB, 30 min d'installation)

3. Dans Android Studio → **Tools** → **SDK Manager** :

- **Android SDK Platform 34** (Android 14)
- **Android NDK 27** (Side panel → SDK Tools)
- **Android SDK Build-Tools 34**

4. **Variables d'environnement** :

- Windows (PowerShell admin) :

```
setx ANDROID_HOME "C:\Users\admin\AppData\Local\Android\Sdk"  
setx NDK_HOME "%ANDROID_HOME%\ndk\27.0.12077973"
```

- macOS / Linux (`~/.zshrc` ou `~/.bashrc`) :

```
export ANDROID_HOME=$HOME/Library/Android/sdk  
export NDK_HOME=$ANDROID_HOME/ndk/27.0.12077973
```

5. Redémarre le terminal.

Initialiser le projet Android

```
cd apps/frontend  
npx @tauri-apps/cli android init
```

Cela crée `src-tauri/gen/android/` (un projet Android Studio complet).

Build de test (APK installable directement)

```
npx @tauri-apps/cli android build --apk
```

Résultat : `apps/frontend/src-tauri/gen/android/app/build/outputs/apk/universal/release/app-universal-release.apk`

Tester sur ton téléphone

Méthode 1 — Câble USB

1. Active le **mode développeur** sur ton téléphone : Paramètres → À propos → tape 7 fois sur « Numéro de build »
2. Active **Débogage USB** dans Options développeur
3. Branche le téléphone à ton PC
4. Dans le terminal :

```
adb install app-universal-release.apk
```

Méthode 2 — Email / WhatsApp / Drive

1. Envoie le fichier `.apk` à toi-même sur ton téléphone
2. Ouvre le fichier → autorise « Installer depuis cette source »
3. Installation

Publier sur le Google Play Store

1. **Créer un compte Play Console** : <https://play.google.com/console> — paiement unique de **25 \$**
2. **Signer l'APK** avec une clé persistante (à conserver précieusement, sinon impossibilité de mettre à jour) :

```
keytool -genkey -v -keystore athenis-release.keystore \  
-keyalg RSA -keysize 2048 -validity 10000 \  
-alias athenis
```

3. **Builder un AAB** (Android App Bundle, format requis par Google) :

```
npx @tauri-apps/cli android build --aab
```

4. Upload sur le Play Console + remplir la fiche (description, captures, politique de confidentialité, classification)

5. Délai d'approbation Google : **1 à 7 jours** la première fois

Avantages Android natif

- ☒ Apparaît dans le **Play Store** → utilisateurs te trouvent par recherche
- ☒ Mises à jour automatiques via Play Store
- ☒ Notifications push (avec Firebase Cloud Messaging)
- ☒ Accès caméra (scan factures), GPS, fichiers

Limitations

- ⚠ Compte Play Console : 25 \$ une fois
- ⚠ Politique de confidentialité obligatoire (peux générer sur <https://app-privacy-policy-generator.firebaseapp.com>)
- ⚠ Re-signature à chaque mise à jour
- ⚠ Tauri Mobile est récent — quelques bugs possibles

18.D — Application iOS (Tauri Mobile) 🍏

Contraintes Apple incontournables

- **Mac obligatoire** (Xcode ne tourne pas sous Windows/Linux)
- **Compte Apple Developer** : **99 €/an** (pas d'option gratuite pour la publication)
- **Signature avec certificat Apple** obligatoire
- **App Review** : 1-7 jours, ils inspectent l'app en détail

Pré-requis (sur le Mac)

1. **macOS Ventura 13+** (Sonoma recommandé)
2. **Xcode 16+** : Mac App Store (gratuit, mais 15 GB)
3. **CocoaPods** : `brew install cocoapods`
4. **Rust** déjà installé (voir 18.B)
5. **Compte Apple Developer** validé : <https://developer.apple.com/programs/>

Initialiser iOS

```
cd apps/frontend
npx @tauri-apps/cli ios init
```

Tester sur ton iPhone

1. Ouvre `apps/frontend/src-tauri/gen/apple/Athenis.xcodeproj` dans Xcode
2. Branche ton iPhone au Mac
3. En haut, choisis « **Athenis** » → **ton iPhone**
4. Clique ► **Run** (triangle)
5. Sur l'iPhone, autorise le développeur dans Réglages → Général → VPN et gestion d'appareils

Build pour publication

Dans Xcode :

Product → **Archive** → **Distribute App** → **App Store Connect** → **Upload**

Ensuite, dans <https://appstoreconnect.apple.com/> :

1. Crée la fiche app (nom, descriptions, captures iPhone et iPad, mots-clés)
2. Soumets l'archive pour review
3. Patiente 1-7 jours

Coûts iOS récapitulatifs

Item	Coût
Mac (si tu n'en as pas)	700-2000 €
Apple Developer Program	99 €/an
Xcode	0 € (gratuit)

18.E — Stratégie de versioning recommandée

Pour gérer les versions Web, Desktop, Android, iOS de façon cohérente :

Composant	Version dans	Mettre à jour comment
Web (PWA)	<code>apps/frontend/package.json</code> + Service Worker	Auto à chaque déploiement
Desktop Tauri	<code>apps/frontend/src-tauri/tauri.conf.json</code> (<code>version</code>)	Manuel + re-build + nouveau release GitHub
Android	<code>apps/frontend/src-tauri/gen/android/app/build.gradle.kts</code> (versionCode + versionName)	Manuel + re-build + nouveau release Play Store
iOS	Xcode → cible → Build Settings (Version + Build)	Manuel + nouveau Archive + upload TestFlight/App Store

Convention recommandée : suivre la version du backend (ex. `1.2.3`). Quand tu mets à jour le backend, mets à jour les 4 versions en même temps.

18.F — Récapitulatif et ordre de priorité

Pour démarrer en douceur :

1. **PWA**  (déjà actif) — couvre 95 % des cas, 0 effort
2. **Tauri Desktop Windows** ensuite, si tu vises un public Windows pro qui veut une vraie appli
3. **Tauri Android** ensuite, quand tu auras des utilisateurs nomades
4. **iOS App Store** dernier, uniquement si tu vises un marché iPhone professionnel

Tu peux **commencer uniquement avec la PWA** et ajouter les autres modes progressivement, sans modifier le serveur.

19. Phase 17 : dépannage — problèmes fréquents {#19-phase-17--depannage}

18.1 — Le site ne s'ouvre pas du tout

```
docker compose -f infra/docker-compose.yml ps
```

Statut	Solution
<code>nginx</code> n'est pas <code>Up</code>	<code>docker compose logs nginx</code> — souvent un problème de certificat
<code>backend</code> redémarre en boucle	<code>docker compose logs backend</code> — vérifie le <code>.env</code>
<code>db</code> n'est pas <code>healthy</code>	Vérifie <code>POSTGRES_PASSWORD</code> dans <code>.env</code>

18.2 — "Connection refused" sur le navigateur

- Vérifie le pare-feu : `sudo ufw status` (ports 80/443 doivent être autorisés)
- Vérifie le DNS : `nslookup ton-domaine.com`

18.3 — "Certificat invalide / non sécurisé"

Refais la phase 8 (Let's Encrypt). Si erreur "rate limit", attends 1 heure.

18.4 — "500 Internal Server Error"

```
docker compose -f infra/docker-compose.yml logs --tail=100 backend
```

Cherche les lignes en rouge ou avec "Error". Souvent :

- Migration non appliquée → `docker compose exec backend npx prisma migrate deploy`
- Variable manquante dans `.env`

18.5 — Disque plein

```
df -h
```

Si > 80 % :

```
docker system prune -af --volumes
```

(Supprime les images et volumes non utilisés.)

18.6 — Le VPS ne répond plus du tout

1. Connecte-toi au manager OVH.
 2. **Bare Metal Cloud** → **VPS** → **ton VPS** → **Reboot**.
 3. Patiente 5 minutes puis réessaie SSH.
-

20. Annexes {#20-annexes}

A. Glossaire des termes techniques

Terme	Définition
VPS	Virtual Private Server — un ordinateur en location dans un datacenter
SSH	Secure Shell — protocole de connexion sécurisée à distance
Docker	Outil qui fait tourner des applications dans des "conteneurs" isolés
DNS	Domain Name System — annuaire qui associe un domaine à une IP
HTTPS / SSL	Protocole web sécurisé (chiffré)
Let's Encrypt	Autorité de certification gratuite pour SSL
Cron	Planificateur de tâches Linux (exécute à heures fixes)
Prisma migrate	Outil qui crée/met à jour la structure de la base de données
Backend	Le serveur qui calcule (invisible pour l'utilisateur)
Frontend	L'interface visible dans le navigateur
API	Application Programming Interface — points d'entrée de l'application
PostgreSQL	Base de données relationnelle utilisée par Athenis

B. Checklist finale avant mise en production

- ☐ Domaine acheté et propagé (DNS résout l'IP du VPS)
- ☐ VPS Ubuntu 24.04 LTS commandé et accessible en SSH
- ☐ Utilisateur `athenis` créé (sans connexion root directe)
- ☐ Pare-feu UFW activé (ports 80/443/22)
- ☐ Docker installé et opérationnel
- ☐ Certificat HTTPS Let's Encrypt valide
- ☐ `.env` avec secrets aléatoires forts (JWT_SECRET, ENCRYPTION_KEY 32 chars)
- ☐ Toutes les migrations Prisma appliquées
- ☐ Premier compte super-admin créé et mot de passe **CHANGÉ**
- ☐ SMTP Brevo configuré et un email de test envoyé
- ☐ Backup automatique quotidien planifié (cron)
- ☐ UptimeRobot configuré (alerte SMS/email)

- [] Sentry configuré (capture des erreurs)
- [] Test de bout en bout : connexion, création de facture, comptabilisation
- [] Sauvegarde initiale manuelle effectuée

C. Commandes Linux essentielles

Action	Commande
Voir la charge du serveur	<code>htop</code> (q pour quitter)
Voir l'espace disque	<code>df -h</code>
Voir un fichier	<code>cat fichier</code> ou <code>less fichier</code> (q pour quitter)
Éditer un fichier	<code>nano fichier</code> (Ctrl+O sauve, Ctrl+X quitte)
Lister un dossier	<code>ls -la</code>
Changer de dossier	<code>cd /chemin</code>
Voir mon emplacement	<code>pwd</code>
Voir les conteneurs Docker	<code>docker ps</code>
Logs d'un conteneur	<code>docker logs nom-conteneur</code>
Redémarrer un service Docker	<code>docker compose restart backend</code>
Voir tous les services Docker	<code>docker compose ps</code>

D. Coûts mensuels récapitulatifs

Service	Coût mensuel	Annuel
VPS OVH Value	5,99 €	71,88 €
Backup automatique OVH	2,00 €	24,00 €
Domaine <code>.com</code>	—	8,99 €
Email Brevo (300/j)	0 €	0 €
UptimeRobot	0 €	0 €
Sentry (gratuit jusqu'à 5k erreurs/mois)	0 €	0 €
TOTAL	~8 €/mois	~105 €/an

Pour passer en croissance (1000+ utilisateurs) :

- VPS Essential ou serveur dédié : 30-100 €/mois
- Brevo plan Starter : 25 €/mois (20k emails)

- Sauvegarde externalisée : 5 €/mois

Aide et support

- **Documentation officielle Athenis** : ce dépôt GitHub → fichier `DEPLOY.md`
- **Communauté Docker** : <https://docs.docker.com/>
- **Documentation OVH** : <https://help.ovhcloud.com/>
- **Let's Encrypt** : <https://letsencrypt.org/getting-started/>
- **Stack Overflow** : pour toute question technique en anglais

E. Tableau comparatif Web / Desktop / Mobile

Critère	PWA (Web)	Desktop (Tauri)	Android (Tauri Mobile)	iOS (Tauri Mobile)
Effort initial	0 (déjà actif)	2-4 h par OS	4 h + Play Store	8 h + Mac requis
Effort à chaque MAJ	Auto (re-déploiement web)	Re-build + GitHub Releases	Re-build + Play Store	Re-build + App Store
Coût	0 €	0 € (250 €/an certif. EV optionnel)	25 \$ Play Console une fois	99 €/an Apple Developer
Délai mise en ligne MAJ	Immédiat	Immédiat (téléchargement)	1-7 jours review	1-7 jours review
Hors-ligne	Lecture seule	Complet	Complet	Complet
Stockage utilisateur	~30 Mo cache	~80 Mo	~50 Mo	~50 Mo
Accès caméra/GPS	Limité (Web API)	Oui (API Tauri)	Complet	Complet
Notifications push	Limité (Web Push)	Oui	Oui (Firebase)	Oui (APNs)
Découverte	URL connue ou SEO	GitHub Releases / site	Play Store recherche	App Store recherche
Mac requis	Non	Non (sauf macOS build)	Non	Oui
Recommandé pour	Démarrage, MVP	Pro Windows/Mac	Mobilité Android	Marché iPhone pro

Document généré pour le déploiement d'Athenis — Plateforme financière, comptable et ESG conforme SYSCOHADA / PCG.

Pour le Cameroun et la France.